AUTOMATION IN SOFTWARE DEVELOPMENT — ASSIGNMENT A1

Student Name: Saksham Singh
Roll Number: 24BTRAO035
Course & Semester: Automation in Software Development • V Semester • Unit 2
Assignment: AI Coding Assistant Practicum — Prompt Engineering in Practice
Tool Evaluated: Gemini / Cursor AI Assistant (Free Tier)
Chosen Task: Option C — Task Management REST API with Filtering, Sorting & Pagination
GitHub Repository: https://github.com/sakshamsingh819/AUTOMATION-IN-SOFTWARE-DEVELOPMENT-ASSIGNMENT-A1.git

1. Prompt Engineering Log Table

Step #	Technique	Exact Prompt Used	Tool Response + Student Evaluation
1	**Zero-shot**	Create a lightweight Express.js REST API with in-memory storage for managing Tasks. Include endpoints: POST /tasks, GET /tasks (with filter), GET /tasks/:id, PATCH /tasks/:id, DELETE /tasks/:id.	**Accepted with minor edits.** The assistant generated core Express routing boilerplate and in-memory arrays. Added explicit payload schema checks for PATCH endpoints. Zero-shot suited standard CRUD patterns.
2	**Few-shot**	Generate Express responseHandler.js utility and error middleware implementing unified { success, data, error, meta } envelope pattern. [Provided 3 concrete examples: success payload, 400 validation error, and 404 not found envelope].	**Accepted as-is.** Strictly adhered to the specified JSON schema and generated AppError classes without key-naming drift. Few-shot was necessary to standardize response envelopes across all routes.
3	**Chain-of-thought**	We have a complex query logic requirement for GET /tasks: multi-status filter, full-text search across title/description, custom priority weight sorting (high > medium > low), and pagination. Think step by step through extraction, filtering, text search, custom comparator, and pagination slicing.	**Accepted as-is.** Assistant decomposed the pipeline into 5 phases. Solved non-alphabetical sorting by constructing numeric priority map ({ high: 3, medium: 2, low: 1 }), preventing alphabetical sort bugs.
4	**Zero-shot**	Generate a complete input validation middleware for POST /tasks validating that title is a non-empty string, status is within [pending, in_progress, completed], priority is within [low, medium, high], and	**Accepted as-is.** Generated lightweight validator middleware returning 400 with descriptive error messages. Zero-shot was suitable because field-level

Step #	Technique	Exact Prompt Used	Tool Response + Student Evaluation
		dueDate is an ISO8601 string.	validation is unambiguous.
5	**Few-shot**	Generate Jest and Supertest unit/integration test suites for the tasks endpoints. [Provided 2 test examples showing success response structure assertion and 404 envelope validation].	**Accepted as-is.** Assistant created comprehensive test assertions matching our exact { success, data, error, meta } envelope structure, testing edge cases like invalid pagination query params.
6	**Chain-of-thought**	When running GET /tasks?status=pending,completed&page=1&limit=10, the response returns an empty data array despite tasks existing. Think step by step to identify the query parsing bug and resolve it.	**Accepted as-is.** Assistant traced that comma-separated query string values were treated as literal strings rather than split arrays. Resolved by adding string splitting and trimming logic.

2. Prompt Execution Evidence (Screenshots)

Step 1: Zero-Shot Initial Scaffolding

[Screenshot Image: Screenshot From 2026-08-30 08-54-31.png]

Figure 1: Zero-shot prompt generation for Express.js in-memory REST API.

Step 2: Few-Shot Response Envelope Standardization

[Screenshot Image: Screenshot From 2026-08-30 08-55-04.png]

Figure 2: Few-shot prompt enforcing unified JSON envelope pattern and custom error classes.

Step 3: Chain-of-Thought Query Pipeline & Priority Logic

[Screenshot Image: Screenshot From 2026-08-30 08-55-57.png]

Figure 3: Chain-of-thought prompt reasoning through multi-attribute query parsing and weighted priority sorting.

3. Comparative Tool Reflection: AI Assistant vs. GitHub Copilot (~400 Words)

Evaluating the tested AI assistant alongside GitHub Copilot across the development of this REST API highlighted distinct structural and operational strengths in practical prompt engineering workflows. While GitHub Copilot functions primarily as an inline autocomplete engine within single active editor tabs, modern conversational and agentic assistants operate with a broader systemic understanding of application architecture, query parsing, and cross-cutting concerns.

During the zero-shot scaffolding phase, both tools demonstrated comparable speed when generating baseline Express routes. However, notable differences emerged during the few-shot prompting phase. Enforcing a

standardized JSON response envelope (`{ success, data, error, meta }`) across an entire project in GitHub Copilot often requires writing inline prompt comments inside each individual controller and middleware file, which frequently results in slight schema hallucinations and key-naming drift. In contrast, the tested AI assistant ingested the structural input/output schema examples in a single context window, successfully generating modular custom error classes, response helper functions, and centralized route middlewares without omitting nested fields or meta timestamps.

The chain-of-thought phase demonstrated the most substantial performance advantage. When tasked with multi-attribute filtering, substring text matching, and custom priority sorting, instructing the model to "think step by step" forced an explicit breakdown of intermediate operations. This reasoning phase prevented a classic algorithmic bug: standard string sorting in JavaScript orders priorities alphabetically (placing `"medium"` ahead of `"high"`), whereas the assistant synthesized a dedicated numeric weight map (`{ high: 3, medium: 2, low: 1 }`) and built a structured aggregation pipeline before returning the controller code. Copilot typically defaults to simple alphabetical `.sort()` routines unless manually guided through prompt iterations.

In terms of developer velocity, inline Copilot completions are faster for immediate single-line syntax suggestions, but conversational agent workflows drastically reduce overall debugging and architectural refactoring time. In a real-world software engineering team, I would select this conversational/agentic tool over GitHub Copilot for greenfield API design, test harness generation, and complex data pipeline refactoring where step-by-step reasoning and project-wide schema consistency are essential. Copilot remains optimal for low-latency, localized typing acceleration in established, mature codebases.